

STRINGS

Chapter 7

Copyright © 2017 STEM Course Prep



CHARACTERS

- **Characters** include:
 - letters of the alphabet
 - digits
 - punctuation marks
 - **Control characters**
 - backspace
 - tab
 - **White space characters**
 - space bar
 - tab
 - **Leading blanks**
 - blank spaces at the beginning of a string,
 - **Trailing blanks**
 - blank spaces at the end of a string

STRINGS

- **String** = any number of characters contained in single quotes
 - Think of text
- **Substring** = a subset or portion of a string
- String with length 0 = empty string
 - Example: `' '`
- String variables can be created using
 - assignment statement
`fruit='apples and oranges'`
 - input function
`fruit=input('Enter your favorite fruit','s')`

STRINGS

- Strings are treated as vectors in MATLAB
 - Each character is an element of the vector
- Several built-in functions and operators that work with numeric vectors work with strings also
 - transpose or `'`
 - `size`
 - `length`
- Need to pull out certain elements or substrings?
 - Use indexing
 - Can index into a string just like you did with numeric matrices

CONCATENATION

- Sometimes we need to combine vectors/matrices
- We do this using concatenation
- 2 methods to concatenate strings:
 - Horizontal concatenation
 - Vertical concatenation

HORIZONTAL CONCATENATION

- Horizontal concatenation
 - Puts all characters in one row – think of creating a long sentence
 - 2 ways to do it:
 - Use `[]` – each string goes in single quotes


```
>> ['Austin, ' ' Texas']
ans =
      'Austin, Texas'
```
 - Using **strcat**

```
>> strcat('Austin, ', ' Texas')
ans =
      'Austin, Texas'
```
 - What's the difference between the methods?
 - `[]` retains trailing blanks
 - **strcat** will not retain trailing blanks

VERTICAL CONCATENATION

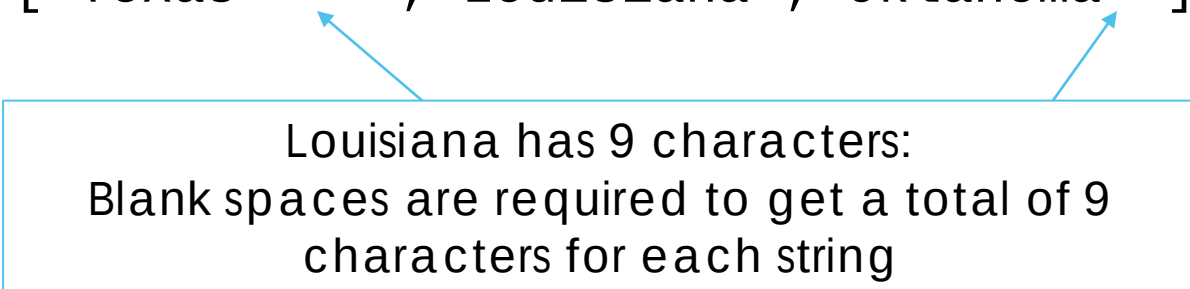
- Think of putting strings in rows on top of each other

Texas
Louisiana
Oklahoma

- Creates a *character matrix*
 - Each single character is an element in the matrix
- How do you do it?
 - Use [] and separate with semicolons
 - ; indicates end of row
 - Built-in function **char**
 - **Note:** you are creating a matrix – # of characters on each row must be the same
 - Use blank spaces to make sure all strings have the same number of characters
 - **char** does this automatically

VERTICAL CONCATENATION

- Example: Vertically concatenate the strings Texas, Louisiana, Oklahoma
- Method 1: brackets w/ ;
`cmat = ['Texas'; 'Louisiana'; 'Oklahoma'];`



Louisiana has 9 characters:
Blank spaces are required to get a total of 9
characters for each string

- Method 2: char
`cmat = char('Texas', 'Louisiana', 'Oklahoma');`

VERTICAL CONCATENATION

- Character matrix produced:

T	e	x	a	s				
L	o	u	i	s	i	a	n	a
O	k		a	h	o	m	a	

- Each character is an element in the matrix

```
char1=cmat(1,3)
>>char1 =
    'x'
```

```
char2=cmat(2,8)
>>char2 =
    'n'
```

```
char3=cmat(3,9)
>>char3 =
    ' '
```

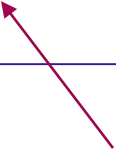
SPRINTF

- Recall what `fprintf` does
 - Allows user to create formatted print statements
 - Statements change when values of the variables change
 - Displays statement to the screen – doesn't save it
- **`sprintf`** is very similar to `fprintf`
 - Main differences
 - String is created
 - String can be saved with a variable name
- Why would we want this?
 - Customized strings can be created and used as inputs into functions requiring strings
 - Title
 - Legend
 - Input
 - Etc.

SPRINTF

```
state=input('Enter the state you were born in:\n','s');  
stg=sprintf('Name the %s city in which you were born:\n',state);  
city=input(stg,'s');
```

```
Enter the state you were born in:  
TX  
Name the TX city in which you were born:  
Dallas
```



State will change based on user input

SPRINTF¹²

```
% Create custom labels for plots
```

```
ang=0:1:365;
```

```
vel=100*cosd(ang);
```

```
%find data to be used in plot
```

```
min_ang=min(ang);
```

```
max_ang=max(ang);
```

```
minV=min(vel);
```

```
maxV=max(vel);
```

```
plot(ang,vel,'mp')
```

```
axis tight
```

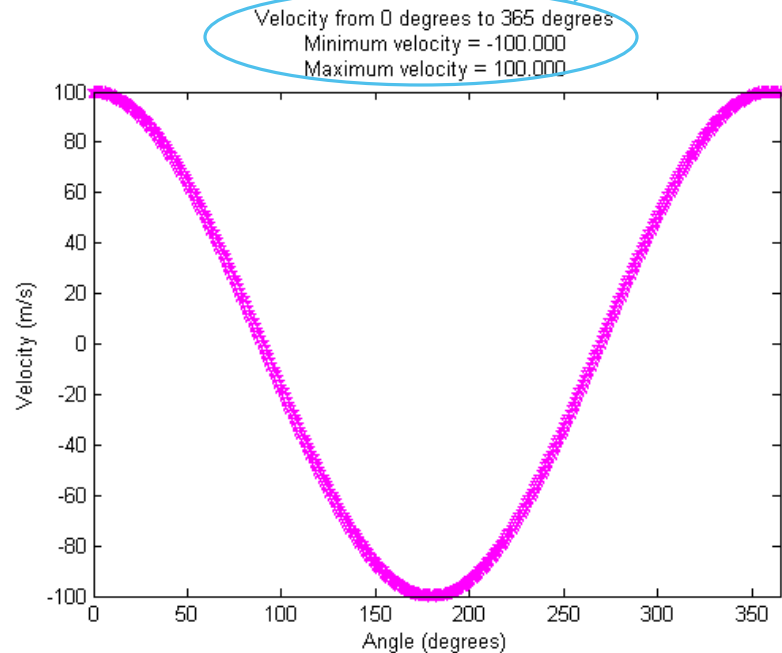
```
xlabel('Angle (degrees)')
```

```
ylabel('Velocity (m/s)')
```

```
string=sprintf('Velocity from %d degrees to %d degrees\nMinimum  
velocity= %.3f\nMaximum velocity =  
%.3f',min_ang,max_ang,minV,maxV)
```

```
title(string)
```

Values automatically update if
velocity equation is changed



STRING FUNCTIONS

Need to remove blank spaces?

- **deblank** removes trailing blanks

```
str='Hello   ';  
deblank(str)
```



```
ans =  
    'Hello'
```

- **strtrim** removes leading and trailing blanks

```
str='   Hello   ';  
strtrim(str)
```



```
ans =  
    'Hello'
```

- Is there a function to remove spaces from the middle of a string? No

Need to add blank spaces?

- **blanks** creates a string of all blank spaces (can be useful in creating blank space in output by transposing; e.g. if 5 blank lines are desired, print `blanks(5)`)

STRING FUNCTIONS

Need to add blank spaces?

- **blanks** creates a string of blank spaces

```
mat=['a';blanks(2)';'d']
```

```
mat =  
4x1 char array  
 'a'  
  ' '  
  ' '  
 'd'
```

```
str=['1',blanks(5),'2']
```

```
str =  
 '1      2'
```

STRING FUNCTIONS

- Need to change the case of text?
 - **upper** converts a string to all upper case letters

```
word='Monday';  
upper(word)
```



```
ans =  
    'MONDAY'
```

- **lower** converts a string to all lower case letters

```
word='TUESDAY';  
lower(word)
```



```
ans =  
    'tuesday'
```

STRING COMPARISONS

When comparing strings, don't use ==

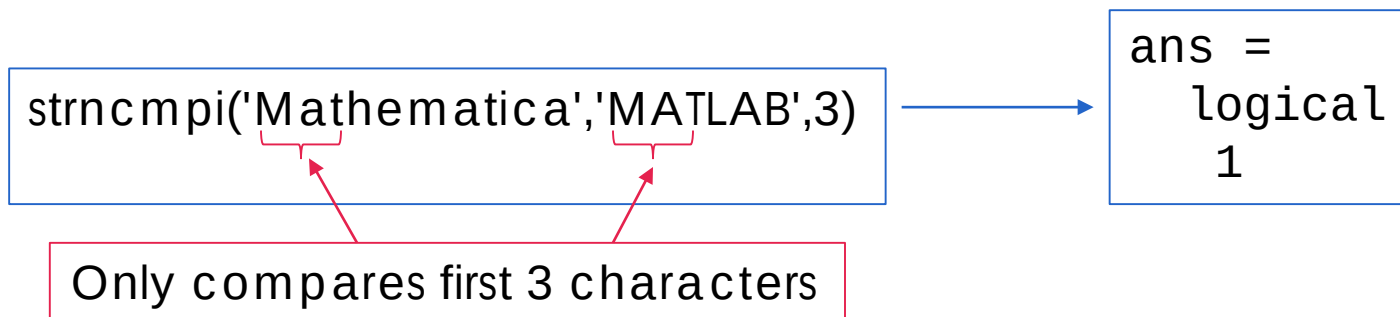
- Use built-in string functions listed below
- **strcmp** compares two strings
 - returns a logical
 - 1 identical
 - 0 not identical
 - Remember
 - Case-sensitive
 - Blank spaces are characters

```
strcmp('San Diego', ' San Diego')
```

```
ans =  
    logical  
     0
```

STRING COMPARISONS

- **strncmp** compares the first N characters of the strings
`strncmp(string1, string2, N)`
- **strcmpi** ignores case
`strcmpi(string1, string2)`
- **strncmpi** compares first n characters, also ignores case
`strncmpi(string1, string2, N)`



STRFIND

- **strfind(string, pattern)** – locates all occurrences of the pattern in the string
 - Returns a vector containing starting indices of the pattern
 - Empty vector returned if pattern not found

```
stg='Houston, Texas';  
strfind(stg, 's')
```

```
ans =  
     4    14
```

index	1	2	3	4	5	6	7	8	9	10	11	12	13	14
	H	o	u	s	t	o	n	,		T	e	x	a	s

```
stg='Houston, Texas';  
strfind(stg, ' ')
```

```
ans =  
     9
```

STRREP

- **strrep(string, oldsubstring, newsubstring)**: replaces all instances of the text `oldsubstring` and replaces it with `newsubstring`
 - the old and new substrings can be different lengths

```
stg='Coke is the best soft drink';  
strrep(stg, 'Coke', 'Dr. Pepper')
```



```
ans =  
    'Dr. Pepper is the best soft drink'
```

LIST DIRECTORY (LS)

- Scenario: You have many sets of data files that you need to load into MATLAB
 - Data files have similar names
- Quick way to do this – use the **ls** function
- ls returns a character array of filenames
- Example: Data files dat1.txt, dat2.txt, dat3.txt, dat4.txt are in the working directory.

```
files=ls ('dat*.txt')
```

* is a wildcard character-
Any file that starts with dat
and ends with .txt gets put
in the array

```
files =  
4x8 char array  
    'dat1.txt'  
    'dat2.txt'  
    'dat3.txt'  
    'dat4.txt'
```

LIST DIRECTORY

- Once character array of files is created use a for loop to load data

```
[r,c]=size(files)
for i=1:r
    load(files(i,:))
end
```

All data files have now been loaded into the workspace

STRtok

- **strtok** – splits a string into two new separate strings
[string1, string2]=strtok(string)
 - First blank space is used as the default delimiter
 - Blank space goes in the second string

```
string='Mechanical Engineering';  
[first, last]=strtok(string)
```

```
first =  
    'Mechanical'  
last =  
    ' Engineering'
```

- To change the delimiter pass a second input argument
[string1, string2]=strtok(string, delimiter)

```
string='Mechanical Engineering';  
[first, last]=strtok(string, 'a')
```

```
first =  
    'Mech'  
last =  
    'anical Engineering'
```

- **eval** – takes a string and evaluates it as code
- Useful for cases when content is not known in advance
 - User inputs information
 - Data file contains information
- For example:

```
x = 1:5;
fn = input('Do you want the max or min of x? ', 's');
eval(strcat(fn, '(x)'))
```

Do you want the max or min of x? max
ans =
5

We concatenated the user-input string with (x) and then evaluated it like regular code

EXAMPLE

- Can use `eval` instead of `ls` for loading multiple data files
- Let's say we have the following data files:
 - `dat1.txt`
 - `dat2.txt`
 - `dat3.txt`
 - `dat4.txt`

```
for i = 1:4  
    eval(sprintf('load dat%d.txt',i))  
end
```

Evaluates the string like it's code

Creates a string for each i value

IS FUNCTIONS²⁵

- **isletter** – returns 1 if input argument is letter of the alphabet
- **isspace** – returns 1 if input argument is white space character

```
str='a';  
isletter(str)
```

```
ans =  
    logical  
     1
```

```
str2='5';  
isletter(str2)
```

```
ans =  
    logical  
     0
```

```
str3=' ';  
isspace(str3)
```

```
ans =  
    logical  
     1
```

- **ischar** – returns 1 if the input argument is a string, or character vector

```
a=1.0;  
ischar(a)
```

```
ans =  
    logical  
     0
```

STRING CONVERSIONS

- **int2str** - converts an integer to a string
 - Floating point #s get rounded to an integer
- **num2str** - converts a real number to a string
- **str2num** - converts a string of numbers to a number

```
a=12;  
int2str(a)
```

```
ans =  
    '12'
```

```
b=12.5;  
int2str(b)
```

```
ans =  
    '13'
```

```
c=27.1;  
num2str(c)
```

```
ans =  
    '27.1'
```

```
d='12345'  
str2num(d)
```

```
ans =  
    12345
```